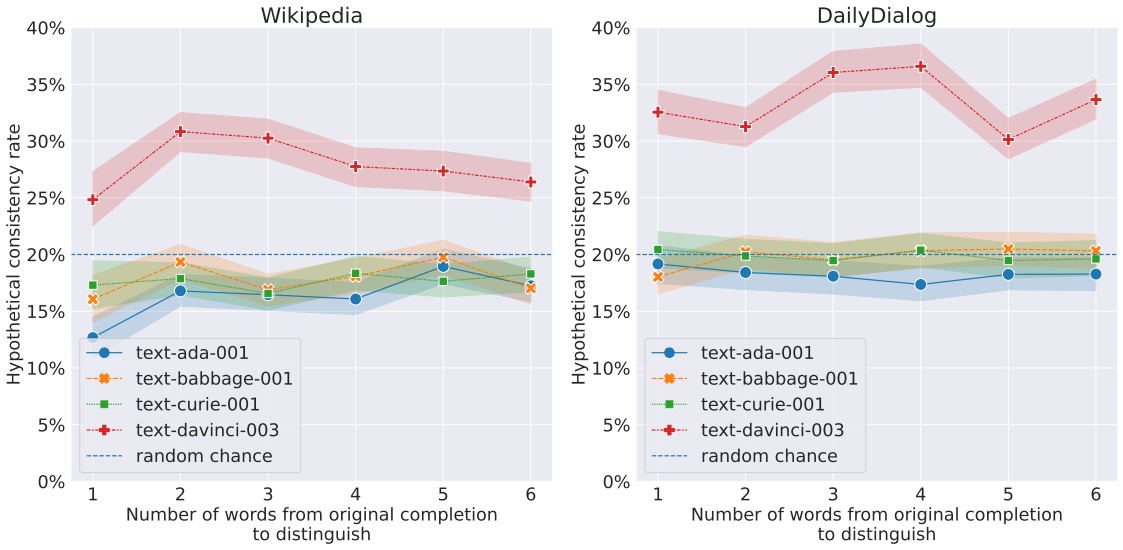

Figure 2: Hypothetical consistency rates on multiple-choice self-knowledge prompts for the Wikipedia and DailyDialog datasets, across the four GPT-3 model sizes. Each line is the average taken across all k-shot prompts, for $k \in [1, \cdots, 10]$. The shaded region represents the 95% confidence interval computed with nonparametric bootstrapping. The label "number of words from original completion to distinguish" corresponds to the quantity m in Table 1.

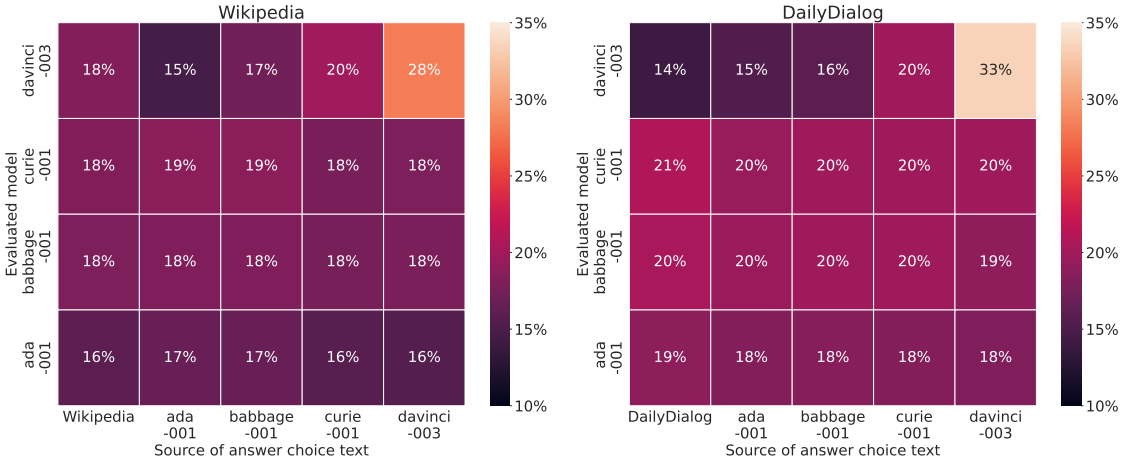

Figure 3: A more detailed breakdown of the numbers in Figure 2: the percentage of the time that each model selects each possible answer choice when prompted with a hypothetical consistency prompt, averaged across all prompts (*i.e.* across all m, the number of words that the model is asked to predict; and k, the number of few-shot examples). The columns labeled "Wikipedia" and "DailyDialog" correspond to the answer choice containing the completion from the original dataset. Model outputs that could not be parsed into an answer choice are not included.

select the answer choice corresponding to its own completion the vast majority of the time, and to only select the other answer choices a negligible proportion of the time. However, for both tasks, only `davinci-003` demonstrates a noticeable preference for its own completion over others. Furthermore, none of the other models display a preference for any other answer choice, including the completions of the other models and the continuation from the original dataset. Despite our use of multiple prompt formats, `ada-001`, `babbage-001`, and `curie-001` all make random choices and cannot predict their own completions.

`davinci-003`'s moderate hypothetical consistency also cannot easily be attributed to dataset memorization, for a couple of reasons. Firstly, we selected only hypothetical consistency prompts for which all five answer choices were distinct – ergo, `davinci-003`'s completion could not have been identical to that of either Wikipedia or DailyDialog. Secondly, we also computed the average percent edit distance (the edit distance divided by the length of the longer string) between the completions of `davinci-003` and the completions of the smaller models and datasets, as shown in Table 2. Across both datasets, the `davinci-003` completions are on average more than 70% different from the dataset completions. Furthermore, the edit distances in Table 2 do not have high variance in each row, indicating that `davinci-003`'s completions are not significantly more different from or similar to one particular source than the others.

4 Evaluating Compositional Self-Consistency

Next, we evaluate compositional self-consistency on the tasks of arithmetic and semantic parsing, since these are tasks for which valid compositional reasoning are important. Both tasks can be framed as computational graphs that are directed and acyclic, with each node having at most one parent – *i.e.* trees. To evaluate compositional consistency, we store the model's answer to the expression represented by each individual sub-tree and generate compositional consistency prompts by creating copies of the original prompts where a single sub-tree expression has been replaced by the model's output for that sub-tree. If a model is compositionally consistent, then it should give the same answer to the original expression as to the copy with the replaced sub-tree. Below, we further describe the experimental setup and give examples for each of the tasks.

4.1 Experimental Setup

We evaluate compositional self-consistency across six models (`ada-001`, `babbage-001`, `curie-001`, `davinci-001`, `davinci-003`, and `gpt4`) and two tasks. Experiments for the first five models were run between Aug. 2022 and Jun. 2023, and experiments for `gpt4` were run between May and Jun. 2023.

Synthetic Arithmetic We generate a set of 400 randomly-nested arithmetic expressions as the initial prompts. We then collect model completions for all possible sub-expressions of each expression using k-shot prompts, with k ranging from 3 to 10. We randomly select the in-context examples for each prompt. The arithmetic expressions have a maximum nesting depth of 5 with a nesting probability of 0.5, and each nested expression is enclosed in parentheses to avoid ambiguity. Operators and operands are randomly selected with uniform probability from the sets $\{+, -, /, \times\}$ and $[1, 2, \cdots, 999]$, respectively.

For example, for the original arithmetic expression "$(2 \times 3) + (6/2)$," we prompt each model with the following three sub-expression prompts, using parentheses to force the correct order of operations:

$p_1 =$ `"Q: 2 × 3 \n A:"`
$p_2 =$ `"Q: 6/2 \n A:"`
$p_3 =$ `"Q: (2 × 3)+(6/2) \n A:"`

For each non-root sub-expression (*i.e.* p_1 and p_2), we then create a new *compositional consistency prompt* by replacing that sub-expression in the original expression (*i.e.* p_3) with the model's completion. For the previous example, if the model answered p_1 and p_2 correctly, this would result in the following two compositional consistency prompts:

$p_{CC}^{(1)} =$ `"Q: 6 + (6/2) \n A:"`
$p_{CC}^{(2)} =$ `"Q: (2 × 3) + 3 \n A:"`

For this example, we then compute a model's *compositional consistency rate* as the proportion of the time that the model's output for p_i is correct, and its outputs for $p_{CC}^{(i)}$ and p_3 are the same.

GeoQuery GeoQuery (Zelle & Mooney, 1996) is a semantic parsing dataset consisting of 880 natural language questions about US geography, paired with FunQL (Kate et al., 2005) parses of those questions. Similar to the synthetic arithmetic task, we first collect model parses for the spans corresponding

to each sub-parse of a sample of 400 GeoQuery training examples via k-shot prompts, for k ranging from 3 to 10. We randomly select the in-context examples for each prompt. For example, consider the GeoQuery example "Which state has the city with the most population?" with corresponding FunQL `state(loc_1(largest_one(population_1(city(all)))))`. Then two of the initial prompts we create include the following:

p_1 = "Create a FunQL query for the following question: 'Which state has the city with the most population?' A: "

p_2 = "Create a FunQL query for the following question: 'city with the most population' A: "

where each prompt is sourced from a non-leaf sub-parse of the original gold FunQL expression (and the other sub-parses are omitted here for brevity).

Since evaluating whether $g_{p_\theta}(f(p', p)) \sim g_{p_\theta}(f(p', g_{p_\theta}(p))$ would involve interleaving natural language with FunQL in this case, we instead measure compositional consistency as the instances where the model's parse for p_2 is correct (*i.e.* $p_2 \sim g_{p_\theta}(p_2)$) and is a sub-parse of its parse for p_1 (regardless of whether the parse for p_1 is correct). This second condition is a slightly more relaxed version of the condition that $g_{p_\theta}(f(p', p)) \sim g_{p_\theta}(f(p', g_{p_\theta}(p))$, where we instead assess whether $g_{p_\theta}(p)$ is being used in $g_{p_\theta}(f(p', p))$. Due to this relaxation, the rate we measure here is an upper bound on the true compositional consistency rate.

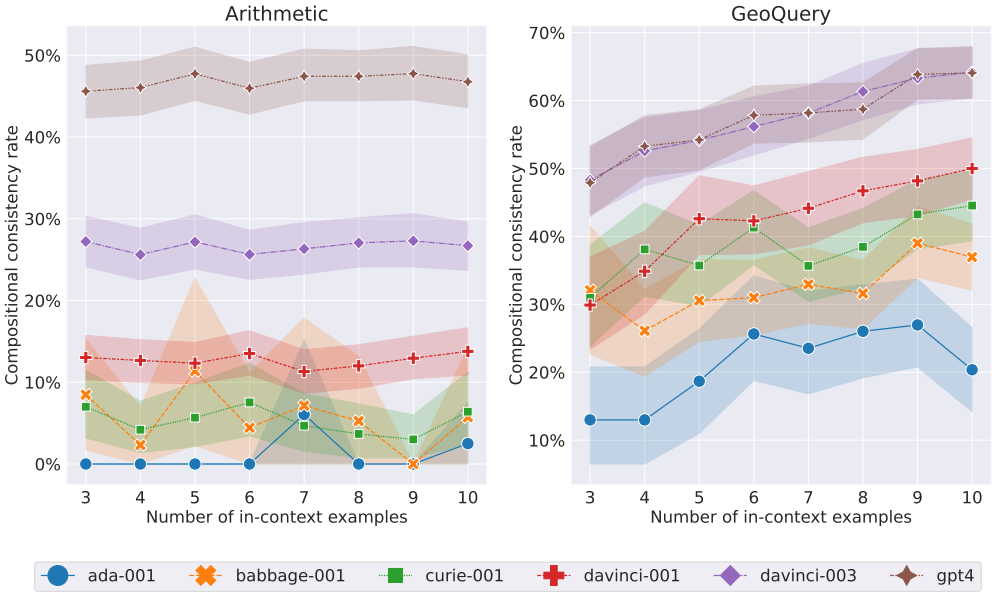

Figure 4: Compositional consistency rates versus the number of in-context examples on the arithmetic and GeoQuery tasks. The shaded region represents the 95% confidence interval computed with nonparametric bootstrapping.

4.2 Results

The compositional consistency rates for all six models are shown in Figure 4. While `davinci-003` and `gpt4` exhibit the highest compositional consistency rates, both are compositionally consistent less than 50% of the time on the arithmetic task and less than 65% of the time on the semantic parsing task. However, all models appear to improve in compositional consistency on the GeoQuery task as the number of in-context examples increases. Furthermore, `gpt4` exhibits significantly more compositional consistency on the arithmetic task than even `davinci-003`. Taken together, these results suggest that both increasing the model capacity and the number of in-context examples can offer some improvements in compositional consistency.